

MQTT Live Dashboard

Technical Documentation v1.0.0

1. Overview

MQTT Live Dashboard is a real-time IoT data visualization application written in Go. It connects to public MQTT brokers, publishes simulated sensor data under persistent topics, subscribes to incoming readings, and displays them on an interactive web interface. The application includes a built-in MQTT broker for local testing and development.

The system architecture consists of four primary components: an MQTT client for external broker communication, a publisher that generates synthetic sensor data, a subscriber that ingests and stores readings, and an HTTP/WebSocket server that serves the dashboard to browsers. A custom lightweight MQTT broker runs on port 1883 and handles basic CONNECT, SUBSCRIBE, PUBLISH, PING, and DISCONNECT operations.

2. System Requirements

2.1 Hardware Requirements

Minimum: 512 MB RAM, 1 CPU core, 100 MB disk space. Recommended: 1 GB RAM, 2 CPU cores, 200 MB disk space. The application is lightweight and runs on resource-constrained environments including Raspberry Pi and cloud VPS instances.

2.2 Software Requirements

Go 1.22 or newer. Git for source control. Optional: mosquitto clients for broker testing. Optional: systemd for service management on Linux.

2.3 Supported Platforms

Linux amd64, Linux arm64, Windows amd64, Windows arm64, macOS amd64 (Intel), macOS arm64 (Apple Silicon), FreeBSD amd64.

3. Installation

3.1 From Source

Clone the repository and build the binary.

```
git clone https://github.com/YOUCENgg/mqtt-dashboard.git
cd mqtt-dashboard
```

```
go mod tidy
go build -o mqtt-dashboard cmd/dashboard/main.go
```

3.2 Cross-Compilation

Build for Windows from Linux or macOS.

```
GOOS=windows GOARCH=amd64 go build -o mqtt-dashboard.exe cmd/dashboard/main.go
```

Build for macOS from Linux.

```
GOOS=darwin GOARCH=arm64 go build -o mqtt-dashboard-mac cmd/dashboard/main.go
```

3.3 Pre-built Binaries

Download the appropriate binary from the GitHub Releases page. Extract if necessary. Make executable on Unix systems with `chmod +x mqtt-dashboard`.

4. Configuration

Configuration is stored in `config.yaml` in the application root directory. The file uses YAML format and supports the following fields.

```
server:
  host: 0.0.0.0
  port: 8081
mqtt:
  broker: tcp://broker.hivemq.com:1883
  client_id: youcefng-dashboard
  username: ""
  password: ""
  topics:
    - name: temperature
      topic: rmbtech/interview/rug/NegadYoucef/sensors/temperature
      interval: 3s
      min_value: 20.0
      max_value: 35.0
      unit: C
```

The `server` block controls the HTTP dashboard server. The `mqtt` block configures the external broker connection. The `topics` array defines which topics to publish and subscribe to. Each topic has a name for display, a full MQTT topic path, a publishing interval in Go duration format, min and max values for random data generation, and a unit string for display.

5. Running the Application

5.1 Basic Execution

```
./mqtt-dashboard
```

The application starts two network services. The dashboard HTTP server listens on the configured host and port. The built-in MQTT broker listens on TCP port 1883. Logs are written to stdout.

5.2 Expected Output

```
Starting built-in MQTT broker on port 1883
Built-in MQTT broker starting on port 1883
Subscribing to: rmbtech/interview/rug/NegadYoucef/sensors/temperature
Connected to MQTT broker
Dashboard starting on http://0.0.0.0:8081
HTTP server starting on 0.0.0.0:8081
```

5.3 Accessing the Dashboard

Open a web browser and navigate to <http://localhost:8081>. The dashboard loads automatically. Data appears within five to ten seconds of application startup, depending on the configured topic intervals.

6. Architecture

6.1 Data Flow

1. The Publisher goroutine generates random sensor values at configured intervals.
2. Each value is serialized to JSON and published to the external MQTT broker.
3. The Subscriber goroutine receives messages from the same topics.
4. Incoming messages are parsed, timestamped, and stored in MemoryStorage.
5. MemoryStorage broadcasts each new reading to all WebSocket subscribers.
6. Connected browsers receive the update via WebSocket and redraw the chart.

6.2 Component Diagram

```
Publisher -> HiveMQ Broker -> Subscriber -> Storage -> WebSocket -> Browser
|
v
Built-in Broker (port 1883)
```

6.3 Package Structure

```
cmd/dashboard/main.go Application entry point and wiring
internal/mqtt/client.go MQTT client wrapper for external brokers
internal/mqtt/publisher.go Synthetic data generator and publisher
internal/mqtt/subscriber.go Message receiver and parser
internal/mqtt/broker.go Custom lightweight MQTT broker
internal/server/http.go HTTP server and route registration
internal/server/handlers.go REST API endpoint implementations
```

```
internal/server/websocket.go WebSocket connection manager
internal/storage/interface.go Storage abstraction
internal/storage/memory.go In-memory ring buffer implementation
internal/config/config.go YAML configuration loader
internal/models/models.go Core data structures
```

7. Built-in MQTT Broker

7.1 Capabilities

The custom broker implements a subset of MQTT 3.1.1. Supported packet types: CONNECT, CONNACK, SUBSCRIBE, SUBACK, PUBLISH, PUBACK, PINGREQ, PINGRESP, DISCONNECT. QoS 0 is fully supported. QoS 1 is partially supported. QoS 2 is not implemented. Retained messages are not supported. Last Will and Testament is not supported. Authentication is not implemented. All connections are accepted.

7.2 Testing the Broker

Terminal 1. Subscribe to a topic.

```
mosquitto_sub -h localhost -p 1883 -t test/topic
```

Terminal 2. Publish a message.

```
mosquitto_pub -h localhost -p 1883 -t test/topic -m hello
```

The message hello should appear in Terminal 1. This confirms the broker accepts connections, handles subscriptions, and routes publishes correctly.

8. API Reference

8.1 REST Endpoints

```
GET /health
Returns application status. Response: {status: healthy}

GET /api/topics
Returns list of monitored topics. Response: [string]

GET /api/latest?topic=TOPIC
Returns most recent reading for a topic. Response: SensorData JSON

GET /api/history/TOPIC
Returns historical data points. Response: [SensorData]
```

8.2 WebSocket Protocol

```
Endpoint: ws://HOST:PORT/ws

Client to Server: No messages required.

Server to Client (metric update):
{
```

```
type: metric,  
topic: rmbtech/interview/rug/NegadYoucef/sensors/temperature,  
value: 25.3,  
unit: C,  
timestamp: 2026-08-04T22:30:00Z  
}
```

9. Frontend Features

9.1 Dashboard Components

Connection status indicator. Green dot when WebSocket is connected. Red dot when disconnected. Metric cards showing current value, unit, topic path, and last update time. Topic toggle buttons to show or hide specific datasets on the chart. Live line chart with up to 30 data points per topic. Statistics cards displaying minimum, maximum, and average values per topic.

9.2 Chart Controls

Clear button removes all data points and resets the chart. Grid button toggles background grid lines. Topic toggles add or remove datasets from the display without affecting data collection.

10. Troubleshooting

10.1 Port Already in Use

If port 8081 is occupied, change the port in config.yaml. If port 1883 is occupied, the built-in broker will fail to start. Run as root only if necessary. Prefer using ports above 1024.

10.2 Broker Connection Refused

Verify the external broker URL is correct. Check network connectivity. Ensure firewall rules allow outbound TCP on port 1883. The default broker is broker.hivemq.com which is publicly accessible.

10.3 Dashboard Shows No Data

Wait at least one interval cycle. Check browser console for WebSocket errors. Verify the publisher is running by subscribing to the public topic with `mosquitto_sub`. Check application logs for connection errors.

10.4 WebSocket Disconnects

Check for proxy or firewall interference. The WebSocket endpoint is at `/ws` on the same host and port as the HTTP server. Ensure no reverse proxy is blocking WebSocket upgrades.

11. Deployment

11.1 Systemd Service

Create `/etc/systemd/system/mqtt-dashboard.service`:

```
[Unit]
Description=MQTT Live Dashboard
After=network.target

[Service]
Type=simple
User=mqtt-dashboard
WorkingDirectory=/opt/mqtt-dashboard
ExecStart=/opt/mqtt-dashboard/mqtt-dashboard
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start.

```
sudo systemctl daemon-reload
sudo systemctl enable mqtt-dashboard
sudo systemctl start mqtt-dashboard
```

11.2 Docker

Build image.

```
docker build -t mqtt-dashboard .
```

Run container.

```
docker run -p 8081:8081 -p 1883:1883 -v $(pwd)/config.yaml:/app/config.yaml
mqtt-dashboard
```

This project was built with the help of LLM assistance.